SENTAURUS DOCUMENTATION / LOCAL HTML EXPORT

Optimizer User Guide

W-2024.09

Exported from the installed server documentation on 2026-10-22.

> This is a local PDF export of the complete User Guide chapter tree and copyright page. It is not the publisher's native PDF.
>
> Source installation: W-2024.09-SP1
> Source: manuals/tcad_spice/opt_api/user_guide.html
>
> Chapter links are retained. Links to API pages outside this guide are shown as plain text. The original HTML, assets and API pages are preserved separately in HTML_Sources.zip.

Included sections

Optimizer User Guide
Introduction
Optimization Parameters
Error Calculation
Sampling
Sensitivity and Screening Analysis
Response Surface Models
Copyright

Optimizer User Guide

Welcome to the optimizer User Guide! This documentation provides detailed usage information for the optimizer library.

Contents:

- Introduction
- The Fitter class
  - Jacobian calculation
  - Target functions
  - Least squares backends
  - The minimize backend
  - The DoE backend
  - SHGO
  - Differential Evolution
  - Multi-Objective optimization
- Optimization Parameters
  - OptParam
  - TOptParam
  - LinkedOptParam
  - DOptParam
  - Parameter Space
  - Parameter Transforms
- Error Calculation
  - Residuals
  - Costs
- Sampling
  - SWB Design of Experiments generator
  - Examples
- Sensitivity and Screening Analysis
  - Local Sensitivity Analysis
  - Global Sensitivity Analysis
  - Screening Analysis with Polynomial RSM
- Response Surface Models
  - Gaussian Process Kernels

Introduction

The optimizer library provides a common interface to access different underlying algorithms to perform fittings in different tools such as Garand or Mystic. The main entry point for a user is the `Fitter` class, which provides a series of built-in interfaces or backends that can be selected on creation and give access to different sets of algorithms.

The Fitter class

The `Fitter` can be imported into any python environment as follows:

```
from optimizer.fitter import Fitter
```

The backends available in the Fitter class are:

Interface	Description
`mkl_nls`	Use Intel MKL nonlinear least squares solvers for bounded or unbounded problems.
`least_squares`	Use least squares solvers. Available internal methods are trust region, dogleg and Levenberg-Marquardt.
`minimize`	Use minimisers. A cost function to aggregate the residuals must be given in this case (RMS error by default). Available minimisers include trust region, BFGS (default), L-BFGS-B, COBYLA, etc.
`doe`	Use design of experiments to find a good solution to the problem. It can be static (e.g. grid search) or adaptive (Bayesian optimisation).
`shgo`	This method runs an initial search and local optimizations starting from selected points of the search.
`differential_evolution`	Population based evolutionary algorithm in which an initial set of points is evolved using vector differences between different points.
`pymoo`	Population based evolutionary algorithms for multi-objective optimization from the pymoo library.

Parameters for the optimiser can be set using the `Fitter.set_optimization_parameters` method.

Jacobian calculation

For gradiend-based methods (included in `mkl_nls`, `least_squares`, `minimize` and `shgo`), the options for the calculation of the Jacobian matrix (or gradient vector) are set using `jacobian_options`, which is a dictionary with the following optional fields:

Option	Default	Description
`eps`	`1e-3`	Size of the step for the finite-differencei approximation to the Jacobian.
`step_type`	`"rel"`	Determines how the step (`eps`) is applied. It can be either `"rel"` for a relative value or `"abs"` for an absolute value in the scaled space
`step_method`	`"2-point"`	Describes the method use to calculate the finite-difference approximation. It can be either `"2-point"` (or `"fd"`) for a forward difference approximation or `"3-point"` (or `"cd"`) for a centra difference approximation

Target functions

There are a few things that need to be taken into account when defining a target function:

- By default, target functions are expected to take `OptParam` as input. However, it is possible to use target functions that take arrays instead if we pass the key `wrap=False` to the method `do_fit` .
- When using the Fitter class with most algorithms, the target function will receive multiple inputs to be processed (e.g. all the points needed for the Jacobian calculation), so they must be able to handle more than one input.
- Some methods will pass additional keyword arguments to the target (e.g. with `doe` and `pymoo` backends), so having optional keywod arguments in the definition will allow it to be used with every algorithm.

Examples

This example defines the function $y = a^2 + b^2$ that takes `OptParam` as input:

```
# Define target function taking a list of OptParam
def target(p, **ignore):
    # Make sure it can handle 1D and 2D arrays for parallel
    #  evaluation of Jacobian
    params = np.atleast_2d(p)
    return [[p[0].value**2 + p[1].value**2] for p in params]

# Select Optimization backend as doe
opt = Fitter("minimize")

# Define OptParams to be optimized
a = OptParam("a", 1., -2., 2., 1.)
b = OptParam("a", 1., -2., 2., 1.)

# Perform optimization
result = opt.do_fit([a, b], target)
```

The same function using an array as input would be:

```
# Define target function taking an array
def target_a(p, **ignore):
    # Make sure it can handle 1D and 2D arrays for parallel
    #  evaluation of Jacobian
    params = np.atleast_2d(p)
    return params[:, 0]**2 + params[:, 1]**2

# Select Optimization backend as doe
opt = Fitter("minimize")

# Define OptParams to be optimized
a = OptParam("a", 1., -2., 2., 1.)
b = OptParam("a", 1., -2., 2., 1.)

# Perform optimization
result = opt.do_fit([a, b], target_a, wrap=False)
```

In the case of multi-objective optimization using the `pymoo` backend, the target function will receive the argument `return_individual_errors` set to `True`. This is because it expects not just an array with each of the outputs, but also the values separated by target name. This is the same format used by Sensitivity Analysis:

```
# Define target function for MOO
def mo_target(x, return_individual_errors=True, **kwargs):
    n_shape = len(np.asarray(x).shape)
    x = np.atleast_2d(x)
    f1 = np.array([x_i.value for x_i in x[:, 0]])
    f2 = np.array([x_i[0].value**2 + x_i[1].value**2 for x_i in x])

    if not return_individual_errors:
        # Example to use function with other backends
        if n_shape == 1:
            return f1[0] + f2[0]
        return f1 + f2

    return np.hstack((f1.reshape(-1, 1), f2.reshape(-1, 1))), [[('f1', [f1[i]]), ('f2', [f2[i]])] for i in
range(f1.shape[0])]

  fitter = Fitter("pymoo")
  fitter.set_optimization_parameters(method='NSGAII',
                                     options={"pop_size": 20, "n_offsprings": 20})
  # Define OptParams to be optimized
  a = OptParam("a", 1., 0., 1., 1.)
  b = OptParam("a", 1., 0., 1., 1.)

  # Do optimization
  result = fitter.do_fit([a, b], mo_target)
```

For `pymoo` it is also possible to use plain arrays if `do_fit` is called with argument `wrap=False`:

```
# Define target function for MOO
def mo_target_a(x, return_individual_errors=True, **kwargs):
    n_shape = len(np.asarray(x).shape)
    x = np.atleast_2d(x)
    f1 = x[:, 0]
    f2 = x[:, 0].value**2 + x[:, 1]**2

    if not return_individual_errors:
        if n_shape == 1:
            return f1[0] + f2[0]
        return f1 + f2

    return np.hstack((f1.reshape(-1, 1), f2.reshape(-1, 1))), [[('f1', [f1[i]]), ('f2', [f2[i]])] for i in
range(f1.shape[0])]

fitter = Fitter("pymoo")
fitter.set_optimization_parameters(method='NSGAII',
                                   options={"pop_size": 20, "n_offsprings": 20})
# Define OptParams to be optimized
  a = OptParam("a", 1., 0., 1., 1.)
  b = OptParam("a", 1., 0., 1., 1.)

# Do optimization
result = fitter.do_fit([a, b], mo_target_a, wrap=False)
```

Least squares backends

When using `mkl_nls`, `least_squares`, the following parameters can be used:

- *ftol*: Tolerance for termination by the change of the cost function (relative).
- *xtol*: Tolerance for termination by the change of the parameter (relative).
- *gtol*: Tolerance for termination by the change of the norm of the gradient of the cost function.
- *max_nfev*: Maximum number of function evaluations.
- *verbose*: Verbosity level, which can be from 0 (silent) to 2 (display information every iteration).

Other options specific to each interface are:

Interface	Option	Description
`mkl_nls`	`method`	Choice for bounded (*bounded_trust_region*) or unbounded (*trust_region*) problem.
	`trtol`	Tolerance for termination by the size of the trust region.
	`delta`	Initial size of the trust region.
`least_squares`	`method`	Least squares method. Can be one of ‘trf’, ‘dogbox’ or ‘lm’.
	`loss`	Loss function used in the algorithm. Can be one of ‘linear’, ‘soft_l1’, ‘huber’, ‘cauchy’ or ‘arctan’.

Examples

The following examples illustrate how to create the Fitter object, and change the solver and stopping criteria.

To create a Fitter object with the Intel MKL solvers backend, we would use the following:

```
opt = Fitter("mkl_nls")
```

To change the Optimizer backend from Intel MKL Trust Region to scipy's implementations of least squares, the command would be:

```
opt.set_method("least_squares")
```

To change the solver used internally from TR (deafult solver for *least_squares*) to Levenberg-Marquardt:

```
opt.set_optimization_parameters(method='lm')
```

To set all the tolerances to 1e-3:

```
opt.set_optimization_parameters(ftol=1e-3, xtol=1e-3, gtol=1e-3)
```

The minimize backend

The `minimize` interface provides access to some of scipy's implementations of a variety of minimization algorithms. In this case, the following parameters are used:

- *method*: Minimization method. Defaults to L-BFGS-B. An alternative derivative-free bounded method is COBYLA.
- *tol*: Tolerance for termination. Additional tolerances can be accepted in the `options` dictionary.
- *options*: Dictionary containing method specific options:

Method	Option	Description
common to all methods	`disp`	Verbosity level.
	`maxiter`	Maximum number of iterations.
`BFGS`	`ftol`, `gtol`	Tolerance for termination by the realtive value of the error (ftol) and its derivative (gtol).
	`norm`	Order of the norm: inf takes the largest element, -inf takes the smallest element.
`L-BFGS-B`	`ftol`, `gtol`	Tolerance for termination by the realtive value of the error (ftol) and its derivative (gtol).
	`maxfun`	Maximum number of function evaluations.
	`maxcor`	Maximum number of variable metric corrections used to define the limited memory matrix.
	`maxls`	Maximum number of line search steps per iteration.
`CG`	`gtol`	Stopping criterion based the norm of the gradient.
	`norm`	Order of the norm: inf takes the largest element, -inf takes the smallest element.
`COBYLA`	`rhobeg`	Initial variable range.
	`catol`	Tolerance (absolute) for constraint violations.
	`tol`	Accuracy for the optimization.
`Nelder-Mead`	`fatol`	Stopping criterion based on change in objective.
	`initial_simplex`	Initial simplex as an array with the coordinates of each vertex.
	`maxfev`	Maximum number of function evaluations.
	`xatol`	Stopping criterion based on change of x.
`Powell`	`direc`	Initial set of direction vectors.
	`ftol`	Stopping criterion based on relative change in objective.
	`maxfev`	Maximum number of function evaluations.
	`xtol`	Stopping criterion based on relative change of x.
`SLSQP`	`ftol`	Tolerance for termination for the value of the objectve .
`TNC`	`eta`	Severity of the line search. Must be 0 < eta < 1.
	`ftol`	Stopping criterion by the relative change of the objective function.
	`gtol`	Stopping criterion based on the norm of the projected gradient.
	`maxCGit`	Maximum number of Hessian-vector product evaluations per main iteration.

Method	Option	Description
	`minfev`	Minimum function value estimate.
	`offset`	Value to subtract from each variable.
	`rescale`	Value (in log10) for objectve rescaling: if 0, rescales at each iteration, if a large value, never rescales, if negative, it is set to 1.3.
	`scale`	Scaling factors to apply to variables.
	`stepmx`	Maximum step for the line search.
	`xtol`	Stopping criterion based on relative change of x.

Examples

```
opt = Fitter("minimize")

opt.set_optimization_parameters(method="COBYLA", options={'rhobeg':2.0})

# Define OptParams to be optimized
a = OptParam("a", 1., -2., 2., 1.)
b = OptParam("a", 1., -2., 2., 1.)

# Perform optimization
result = opt.do_fit([a, b], target)

# Print optimization result
print(result)
```

When using gradien-based methods, it is a good idea to set the finite-difference parameters explicitly:

```
opt = Fitter("minimize")

opt.set_optimization_parameters(method="L-BFGS-B",
                                jacobian_options={"eps": 1e-4, "step_type": "rel", "step_method": "2-point"},
                                option={"maxfun": 50})

# Define OptParams to be optimized
a = OptParam("a", 1., -2., 2., 1.)
b = OptParam("a", 1., -2., 2., 1.)

# Perform optimization
result = opt.do_fit([a, b], target)

# Print optimization result
print(result)
```

The DoE backend

The `doe` interface allows the search of the best solution within a (possibly adaptive) set, and no derivatives are needed in this case. The following parameters are possible:

- *method*: Type of design that will be created
- *options*: Dictionary containing method specific options:

Method	Option	Description
`grid`	`Ns`	Number of samples per dimension. If an integer is given, it is used for all dimensions. A tuple with a number of samples per dimension can also be given.
	`boundary`	Specifies whether the design includes the boundary.
	`location`	Selects whether points in each dimension are chosen at equidistant (*equidist*) or random (*random*) locations.
	`midpoint`	Specifies whether the design includes the midpoint.
`latin`	`Ns`	Number of samples.
	`sample_type`	If given, specifies special type of latin hypercube sampling: *low_correlation* generates a number of samplings (given by *tries*) and chooses the one with the lowest correlation. *nolh* generates a nearly orthogonal sampling.
	`scramble`	Whether to use nested binary scrambling on the generated sample.
	`tries`	Number of generated samplings when using *low_correlation* latin hypercube sampling.
`random`	`Ns`	Number of samples.
`sobol`	`Ns`	Number of samples.
	`skip`	Number of points of the sequence to omit. It can be used to generate points incrementally.
`latin`	`Ns`	Number of samples.
	`sample_type`	If given, specifies special type of latin hypercube sampling: *low_correlation* generates a number of samplings ()given by *tries* and chooses the one with the lowest correlation. *nolh* generates a nearly orthogonal sampling.
	`scramble`	Whether to use nested binary scrambling on the generated sample.
	`tries`	Number of generated samplings when using *low_correlation* latin hypercube sampling.
`bayesian_optimiser`	`Ns`	Number of samples to construct the initial model. Sampling is done using a latin hypercube sample.
	`data_noise`	Expected noise of the simulation results.

Method	Option	Description
	`do_plots`	Specifies whether to plot the Gaussian process RSM (1D and 2D slices if there are three or more parameters).
	`hyperparam_mode`	Selects a mode of setting hyperparameters for the kernel. *fixed* (default) retains the initial value throughout. *ml-ii* selects the values that maximize the likelihood after each iteration.
	`include_points`	A list of dictionaries containing the value for optimization parameters that should be included in the initial search in addition to the points generated using latin hypercube sampling.
	`max_iter`	Maximum number of iterations.
	`n_acquisitions`	Number of acquisition function evaluations per iteration. This will be the number of function evaluations per iteration after the initial search.
	`tag`	String to append to the names of plots created.
	`transform`	Transform to apply to the optimization target.
	`batch_size`	Number of candidate points generated for each acquisition function.
	`batch_generator`	Specifies the method used to generate the different candidates of a batch. Options are “constant_liar”, “kriging_believer”, “local_penalization”, “random” and “kdpp”.
	`batch_generator_options`	Options for the selected batch generator. Only the option “lie” is available for the “constant_liar” method. The value of “lie” can be “current_best”, “current_mean”, or “current_worst”.
	`sparse`	Specifies whether a sparse variational Gaussian process is used to drive the Bayesian optimization.
	`inducing_points`	Specifies the number of inducing points when using a sparse Gaussian process.
`turbo`	`Ns`	Number of samples to construct the initial model. Sampling is done using a latin hypercube sample.
	`do_plots`	Specifies whether to plot the Gaussian process RSM (1D and 2D slices if there are three or more parameters).
	`max_iter`	Maximum number of iterations.
	`tag`	String to append to the names of plots created.
	`transform`	Transform to apply to the optimization target.
	`batch_size`	Number of candidate points per iteration.

Method	Option	Description
	`trust_regions`	Number of trust regions used.
	`shared_constraints`	If True, use a single model for the constraints with data from all trust regions. Otherwise, use one model for each trust region.
`cocabo`	`Ns`	Number of samples to construct the initial model. Sampling is done using a latin hypercube sample.
	`max_iter`	Maximum number of iterations.
	`transform`	Transform to apply to the optimization target.
	`batch_size`	Number of candidate points per iteration.
	`acquisition`	Acquisition function to generate new candidate points.

Examples

With the previous definition of the target, not further changes are needed to use the `doe` backend:

```
# Select Optimization backend as doe
opt = Fitter("doe")

# Define OptParams to be optimized
a = OptParam("a", 1., -2., 2., 1.)
b = OptParam("a", 1., -2., 2., 1.)

# Set optimization parameters
opt.set_optimisation_parameters(method="grid", options={'Ns':5})

# Perform optimization
result = opt.do_fit([a, b], target)

# Print optimization result
print(result)
```

To change the method whithin a backend, only a change in the setting of the optimization parameter is needed. For example, to use Bayesian optimization after the previous example,:

```
opt.set_optimisation_parameters(method="bayesian_optimiser", options={'Ns':10, "max_iter": 10})
result = opt.do_fit([a, b], target)
```

SHGO

The `shgo` backend has two stages: first, a sampling of the space is done. Then, a triangulation of these points is done and a local minimization is started at all locally convex points. The options for the local minimization are the same the ones described for the backend `minimize`. Other options are:

Option	Description
`iters`	Number of iterations used to construct the simplicial complex.
`minimizer_kwargs`	Dictionary with options for the local minimization method. Same options as defined for *minimize* backend. Default method is *SLSQP*.
`n`	Number of points used to construct the simplicial complex. At least 2n points will be generated.
`options`	Dictionary with options for the solver. Posible keys are *f_min* (minimum of the target if known), *f_tol* (relative precision in the function value), *maxfev* (limits the maximum number of function evaluations) ,`maxtime` (maximum-allowed running time), *maxiter* (maximum number of sampling iterations), *minimize_every_iter* (whether to do a local minimisation step after every iteration) and *disp* (verbosity level).
`sampling_method`	Sampling method to construct the simplicial complex used to determine the possible local minima. It can be either “sobol” or “halton”. When using “sobol” the number of points will be rounded to the smallest power of 2 greater than n.

Example

The *shgo* backend needs the same options for the local minimization as `minimize`, but they need to be passed in a different way:

```
# Select Optimization backend as shgo
opt = Fitter("shgo")

# Define OptParams to be optimized
a = OptParam("a", 1., -2., 2., 1.)
b = OptParam("a", 1., -2., 2., 1.)

minimizer_options = dict(method="SLSQP", options={"maxiter": 100})
opt.set_optimization_parameters(n=20, sampling_method="sobol",
                                minimizer_kwargs=minimizer_options,
                                options={'disp':True})

# Perform optimization
result = opt.do_fit([a, b], target)

# Print optimization result
print(result)
```

Differential Evolution

The interface `differential_evolution` is also used a black-box optimisation method. An initial sample is evolved based on distances between points. Options are:

Option	Description
`atol`	Absolute tolerance (in standard deviation of the population) for termination.
`disp`	Verbosity level.
`maxiter`	Maximum number of iterations (generations).
`mutation`	Mutation constant. Must be between 0 and 2. It can be given as a tuple with min and max values.
`popsize`	Population size per dimension. The total number of samples per iteration will be *popsize*dim*.
`recombination`	Recombination constant. Must be between 0 and 1.
`seed`	Seed for the random number generator. Used for reproducibility of results.
`strategy`	Differential evolution strategy. Possible values are ‘best1bin’, ‘best1exp’, ‘rand1exp’, ‘randtobest1exp’, ‘currenttobest1exp’, ‘best2exp’ ‘rand2exp’, ‘randtobest1bin’, ‘currenttobest1bin’ ‘best2bin’, ‘rand2bin’ and ‘rand1bin’.
`tol`	Relative tolerance (in standard deviation of the population) for termination.
`updating`	Update strategy for the best solution. Can be either ‘immediate’ (update after every evaluation) or ‘deferred’ (update after every iteration).

Example

```
# Select Optimization backend as differential_evolution
opt = Fitter("differential_evolution")

# Define OptParams to be optimized
a = OptParam("a", 1., -2., 2., 1.)
b = OptParam("a", 1., -2., 2., 1.)

# Perform optimization
result = opt.do_fit([a, b], target)

# Print optimization result
print(result)
```

Multi-Objective optimization

`pymoo` is an interface to some of the multi-objective optimization algorithms provided by the library of the same name. The following parameters are possible:

- *method*: Algotithm to be used: `NSGAII`, `NSGAIII` or `MOEAD`.
- *options*: Dictionary containing method specific options:

Method	Option	Default	Description
common to all methods	`disp`	`0`	Verbosity level.
	`cost`	`{}`	Cost function to be applied to individual targets. This is especially useful if one of the targets is line data, so that it will be aggregated into a single value.
	`n_offsprings`	`pop_size`	Number of new agents generated at each iteration.
	`pop_size`		Number of initial agents to start the search.
	`termination`	`None`	Termination criterion for the optimization.
`MOEAD`	`crossover`	`SBX(eta=20, prob=1.0)`	Simulated binary crossover; available from `pymoo.operators.crossover.sbx.SBX`
	`mutation`	`PM(prob=0.9, eta=20)`	Polynomial mutation; available from `pymoo.operators.mutation.pm.PM`
	`n_neighbors`	`20`	Number of neighboring agents considered when producing offspring.
	`prob_neighbor_mating`	`0.9`	For each agent, probability of choosing the agent to produce offspring from the neighborhood.
`NSGAII`	`crossover`	`SBX(eta=15, prob=0.9)`	Simulated binary crossover; available from `pymoo.operators.crossover.sbx.SBX`
	`mutation`	`PM(prob=0.9, eta=20)`	Polynomial mutation; available from `pymoo.operators.mutation.pm.PM`
`NSGAIII`	`crossover`	`SBX(eta=30, prob=1.0)`	Simulated binary crossover; available from `pymoo.operators.crossover.sbx.SBX`
	`mutation`	`PM(prob=0.9, eta=20)`	Polynomial mutation; available from `pymoo.operators.mutation.pm.PM`

Note

The target function to be optimized must take a list of `OptParam` as an argument. More details can be found in the OptParam reference.

Examples

```
# Set options
fitter = Fitter("pymoo")
fitter.set_optimization_parameters(method='NSGAII',
                                   options={"pop_size": 20, "n_offsprings": 20})
# Define OptParams to be optimized
a = OptParam("p1", .3, 0.0, 1.0, 1.0)
b = OptParam("p2", .7, 0.0, 1.0, 1.0)

# Do optimization
result = fitter.do_fit([a, b], mo_target)

# Print optimization result
print(result)
```

Or, using an array-based target function:

```
# Set options
fitter = Fitter("pymoo")
fitter.set_optimization_parameters(method='NSGAII',
                                   options={"pop_size": 20, "n_offsprings": 20})
# Define OptParams to be optimized
a = OptParam("p1", .3, 0.0, 1.0, 1.0)
b = OptParam("p2", .7, 0.0, 1.0, 1.0)

# Do optimization
result = fitter.do_fit([a, b], mo_target_a, wrap=False)

# Print optimization result
print(result)
```

Optimization Parameters

In order to define an optimization, we need to decide which parameters must be optimized and what values they are allowed to take. In the optimizer library, this is done with a series of classes that provide a variety of functionality:

Name	Description
OptParam	Basic parameter class
TOptParam	Parameter class to define parameters that can use non-linear transforms
DOptParam	Parameter class to define discrete parameters
LinkedOptParams	Parameter class to link values of one parameter to another existing parameter

OptParam

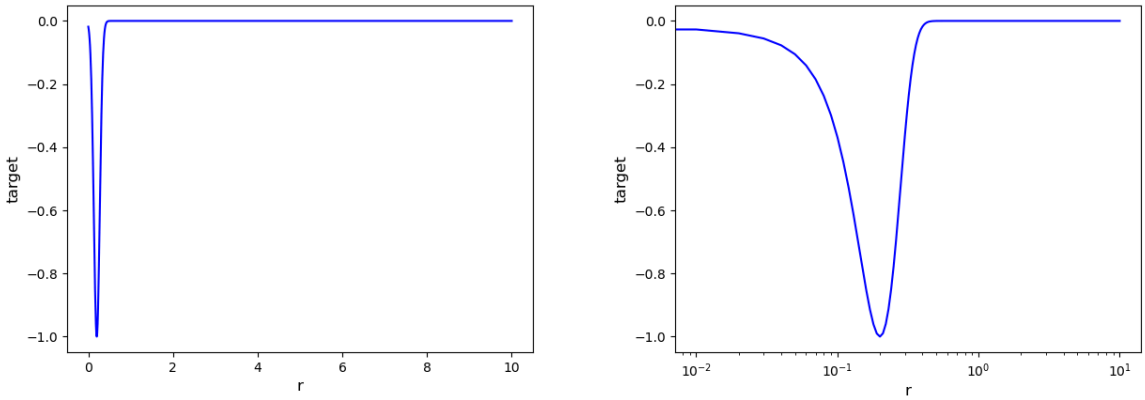
For the simpler problems, `OptParam` is the appropriate class to use. It is used to define continuous parameters that do not require non-linear transforms during the optimization. The following options can be specified when creating an OptParam:

Name	Description
name	Name of the model parameter, e.g. alpha.
default	Default value to use for the parameter.
minval	Minimum value to allow for the parameter. *(maxval required if minval specified)*
maxval	Maximum value to allow for the parameter. *(minval required if maxval specified)*
scale	Scaling factor for the parameter.
precision_rel	Precision for the parameter given as number of significant digits.
precision_abs	Precision for the parameter given as number of decimal places after (positive value) or before (negative value) the decimal point.

Only the name and default value for the parameter are required. Minimum and maximum values, if given, are only applicable when using bounded optimizers. The scaling factor, if provided, is used to scale the absolute parameter values in order to improve the numerical stability of the optimization procedure. By default a scaling factor of 1 is used, although this is not necessarily optimal when extracting parameters that have significantly different orders of magnitude.

TOptParam

In some situations, the dependence of the optimization target on the parameter is such that the problem can be simplified if it is defined in terms of a transform of the parameter. For example, consider the following problem shown linear and log scales:



The basin of the minimum is very narrow, so unless the initial value is close to the final solution, it will be difficult to find it. However, after applying a logarithmic transform to the parameter, the basin of the minimum widens for the same bounds, which makes it easier for the optimizer to find the minimum.

The parameters used to define a `TOptParam` are the same as those for `OptParam` except that instead of a `scale` parameter, there is a `transform` parameter and arguments for the transform can also be added:

Name	Description
transform	Transform to be applied to the parameter during the optimization
**kwargs	Additional keyword arguments to be passed to the parameter transform

There are two ways to define a transform for a TOptParam:

- Using the `ParameterTransform` classes
- Using one of the pre-defined transform strings

For example, to define a parameter with a decimal logarithmic transform, we could do:

```
# Definition using a pre-defined string
tp1 = TOptParam("param1", 0.1, 1e-2, 1e2, transform='decimal-log')

# Definition using a Parameter transform
from optimizer.parameters.transforms import DecimalLogarithmicTransform as Log10
tp1 = TOptParam("param1", 0.1, 1e-2, 1e2, transform=Log10)
```

The transform argument

When `TOptParam` receives a transform, it will behave slightly diferent in the following cases:

- If the transform is an instance of a `ParameterTransform` class, it will use it as-is.
- If the transform is a `ParameterTransform` class, it will be instantiated internall with arguments `lower` and `upper` set to the bounds of the parameter and any other keyword arguments passed to `TOptParam`.
- If the transform is a string, the corresponding `ParameterTransform` class will be instantiated internall with arguments `lower` and `upper` set to the bounds of the parameter and any other keyword arguments passed to `TOptParam`.

The `ParameterTransform` objects will be described in more detailed below, but there are a few things that need to be considered when passing thems as arguments to a `TOptParam` object. For this purpose, we will consider the case of `NormalizeTransform`, `LogarithmicTransform` and `ScaleTransform`, since they exemplify the different cases that can arise in regards to arguments that need to be passed to a `ParameterTransform`:

- `NormalizeTransform` requires arguments `lower` and `upper` which will be mapped to the [-1, 1] range in the transformed space.
- `LogarithmicTransform` does not need any arguments.
- `ShiftTransform` needs a `shift` argument to define the factor used to scale the parameter values.

For example, to use the `NormalizeTransform`, the following three ways are equivalent:

```
from optimizer.parameters.transforms import NormalizeTransform as Normalize

p1 = TOptParam("p_instance", 3, 1, 10, transform=Normalize(1, 10))
p2 = TOptParam("p_class", 3, 1, 10, transform=Normalize
p3 = TOptParam("p_string", 3, 1, 10, transform="normalize")
```

Using multiple transforms

As with single transforms, there are also different ways to ask the `TOptParam` class to compose transforms:

- Using a camposed `ParameterTransform` instance
- Using a list or tuple with `ParameterTransform` instances, classes or strings

The first use case is straightforward:

```
from optimizer.parameters.transforms import ShiftTransform as Shift
from optimizer.parameters.transforms import LogarithmicTransform as Log

t = Log() * Shift(3)
p1 = TOptParam("p_instance", -1, -2, 97, transform=t)
```

The downside of this method is that for transforms that need the `lower` and `upper` arguments, it may be cumbersome as they need to be transformed according to the previous transforms. For example,:

from optimizer.parameters.transforms import NormalizeTransform as Normalize from optimizer.parameters.transforms import DecimalLogarithmicTransform as Log10

t = Normalize(-1, 2) * Log10() # Note that arguments for Normalize are in the log space p1 = TOptParam(“p_instance”, 1, .1, 100, transform=t)

The second use case will instantiate the elements of the list/tuple using *only* arguments `lower` and `upper`, since other arguments could have clashing names (e.g. `ScaleTransform` and `DualLog1pTransform` both have a `scale` argument). This means that transforms that require other arguments such as `Shift` should be instantiated explicitly. For example,:

from optimizer.parameters.transforms import ShiftTransform as Shift from optimizer.parameters.transforms import LogarithmicTransform as Log

p1 = TOptParam(“p_class”, -1, -2, 97, transform=[Log, Shift(3)]) p2 = TOptParam(“p_string”, -1, -2, 97, transform=[“log”, Shift(3)])

The downside we saw for the previous method does not apply in this case, since the bounds are automatically detected. The same example as above would be:

```
from optimizer.parameters.transforms import NormalizeTransform as Normalize
from optimizer.parameters.transforms import DecimalLogarithmicTransform as Log10

p1 = TOptParam("p_class", 1, .1, 100, transform=[Normalize, Log10])
p2 = TOptParam("p_string", 1, .1, 100, transform=["normalize", "decimal-log"])
```

Both syntaxes can be combined so that we can avoid the shortcomings of each indivudual one:

```
from optimizer.parameters.transforms import NormalizeTransform as Normalize
from optimizer.parameters.transforms import ShiftTransform as Shift

t_shift = Shift(3)

p1 = TOptParam("p_mixed_class", -1, -2, 97, transform=[Normalize, t_shift])
p2 = TOptParam("p_mixed_string", -1, -2, 97, transform=["normalize", t_shift])
```

LinkedOptParam

`LinkedOptParams` are used to bind the value of a parameter to another existing parameter, so that they will always take the same value. For example,:

```
# Parameter with a default value of 2.0, bounded between 1.0 and 3.0 and with a scaling factor of 2
 alpha = OptParam("alpha", 2.0, 1.0, 3.0, 2.0)

 # New parameter linked to alpha. It will always have the same value as alpha.
 beta = LinkedOptParam("beta", alpha)

 assert alpha.value == beta.value
```

DOptParam

For discrete optimization parameters that take only a selected list of values, the syntax is slightly different, since they are defined not by minimum and maximum values, but by the list of possible values that it can take (*choices*):

```
p = DOptParam(name, value, choices, sorted=False)
```

The final parameter, *sorted*, specifies whether to consider the listed values as ordered, that is, whether they can be sorted in a meaningful way. This is usually related to the way in which the discrete parameter affects the simulation.

To define a discrete parameter that can take the values [“a”, “b”, “c”, “d”]we would use the following syntax:

```
p = DOptParam("option", "c", choices=["a", "b", "c", "d"])
```

In this example, setting “option” to “a” or “b” represents an unrelated setting (for example, a change in methodology), so using *sorted=False* would be appropriate. However, in some cases, the possible values will define an order. In this case, you would define the discrete parameter as:

```
p = DOptParam("option", "c", choices=["a", "b", "c", "d"], sorted=True)
```

This makes sense, for example, if the setting “option”=”b” reflects an increased value of some parameter with respect to “option”=”a”.

Parameter Space

A *ParameterSpace* is a convenience class used to represent the space generated by a sequence of parameters. It provides utilities for managing points as coordinates in the space instead of having to access the properties per parameter.

The following examples illustrate how to use the *ParameterSpace* class:

```python
# Create parameters that define the space
p1 = OptParam("a", 0, -1, 1)
p2 = TOptParam("b", 1, 0.1, 1000, Log10)
p3 = TOptParam("c", 2, 0, 5, precision_abs=4)
p4 = LinkedOptParam("d", p1)

# Create parameter space
ps = ParameterSpace([p1, p2, p3, p4])

# Transform point. The input can include or exclude linked parameters
x = np.array([0, 10, 3])
xt1 = ps.apply_transforms(x)
x = np.array([0, 10, 3, 0])
xt2 = ps.apply_transforms(x)
assert xt1 == xt2

# The output can also include linked parameters
xt3 = ps.apply_transforms(x, linked=True)
assert xt3[3] == xt3[0]

# Get names of parameters (can include or exclude linked parameters)
names = ps.names(linked=True)

# If you have DoE defined in the unit hypercube, [0, 1]^d, it can be transformed into the parameter space
doe_pspace = ps.points_from_unit_doe(doe, scaled=True)
```

Parameter Transforms

The dependence of target on the optimization parameters can sometimes make the optimization process difficult. For some problems this can be alleviated by doing an optimization on a transformed version of the parameter. This can be achieved using a `TOptParam` together with parameter transforms.

The optimizer library provides the following transforms that can be used when setting a `TOptParam`:

transform	name	arguments	function
DecimalLogarithmicTransform	`"decimal-log"`	N/A	$t(x) = \log_{10}(x)$
DistributionTransform	`"distribution"`	`dist`	$t(x) = dist.cdf(x)$
DualLog1pTransform	`"dual-log1p"`	`scale[=1]`	$t(x) = \begin{cases} s * \log\left(1 + x^{1/s}\right), & \text{if } x \geq 0, \\ -s * \log\left(1 + (-x)^{1/s}\right), & \text{if } x < 0 \end{cases}$
LinearTransform	`"linear"`	`scale[=1]`	$t(x) = x * s$
LogarithmicTransform	`"log"`	N/A	$t(x) = \log(x)$
LogexpTransform	`"log-exp"`	`lower`	$t(x) = \log\left[\exp(x - L) - 1\right]$ $t(x) \sim \begin{cases} x - L, & \text{if } x \to L, \\ \log(x - L), & \text{if } x \to \text{inf}, \end{cases}$

transform	name	arguments	function
LogisticTransform	`"logistic"`	`lower`, `upper`	$t(x) = \log\left(\frac{x-L}{U-x}\right)$
NegativeTransform	`"negative"`	N/A	$t(x) = -x$
NormalizeTransform	`"normalize"`	`lower`, `upper`	$t(x) = 2\frac{x-L}{U-L} - 1$
ReciprocalTransform	`"reciprocal"`	N/A	$t(x) = 1/x$
ScaleTransform	`"scale"`	`scale`	$t(x) = x/s$
ShiftTransform	`"shift"`	`shift`	$t(x) = x + s$
SquareTransform	`"square"`	`center[=0]`	$t(x) = sign(x-c)\cdot(x-c)^2$
SquareRootTransform	`"square-root"`	`center[=0]`	$t(x) = sign(x)\sqrt{\lvert x\rvert} + c$

Note

DistributionTransform's argument `dist` is assumed to be an instance of a scipy's 'frozen' rv_continuous distribution, but only the methods

`cdf`, `ppf`, `stats` and the attribute `name` are used, so any class with these methods could be used instead.

In addition to these, transforms can be combined to produce a new transform. The operator `*` can be used for composing two `ParameterTransform` objects, `t = tN * ... * t2 * t1` is such that $t(x) = (t_N \circ \ldots \circ t_2 \circ t_1)(x) = t_N(\ldots t_2(t_1(x)))$. For example,:

t1 = LinearTransform(2.) t2 = ReciprocalTransform() tc = t2 * t1 assert tc(x) = 1/(2*x)

The operator `~` can be used for inversion,:

t = LinearTransform(2.) it = ~t assert t.inverse(5) == it(5)

The objects constructed using these operators can be passed to a `TOptParam` directly. For example,:

log_neg = DecimalLogarithmicTransform() * NegativeTransform() p = TOptParam(name, -10, -200, -2, log_neg)

Examples

Consider the following problem where the dependence of the target on the parameter is logarithmic:

```
# Target function
def f(x):
    return (np.log10(x[0].value) - 1.5)**2

# Optimization parameters
p1 = OptParam("a", 10, 1e-2, 1e2)

# Set optimization method
opt.set_method("minimize")

result = opt.do_fit([p1], f, jac=True)
```

This simple problem takes eight iterations of the default solver in `minimize`, which is `L-BFGS-B`. To simplify the problem we can do the optimization in a transfomed space:

```
# Target function
def f(x):
  return (np.log10(x[0].value) - 1.5)**2

# Optimization parameters
p1 = TOptParam("a", 10, 1e-2, 1e2, Log10)

# Set optimization method
opt.set_method("minimize")

result = opt.do_fit([p1], f, jac=True)
```

In this case, the solver converges in one iteration since the target is a parabola in the transfomed space.

Error Calculation

The *optimizer* library provides several utitlities for error calculation which are divided in two groups. Residual calculators do a point-by-point comparison of two sequences of data (usually one simulation output and one reference or target data), and cost calculator aggregate a series of residulas into a single value.

Residuals

Residual calculators are used to obtain a (point-wise) distance between some data (*d*) and a target (*t*). The following calculators are available in the *optimizer* library:

residual calculator	name	formula
`residual()` (default)	`"signed"`	$r_i = s_i - t_i$
`squared_residual()`	`"squared"`	$r_i = (s_i - t_i)^2$
`relative_residual()`	`"signed_relative"`	$r_i = (s_i - t_i)/t_i$
`abs_relative_residual()`	`"rel"`	$r_i = \|s_i - t_i\|/t_i$
`abs_residual()`	`"abs"`	$r_i = \|s_i - t_i\|$
`log_relative_residual()`	`"log_rel"`	$r_i = \|(log\|s_i\| - log\|t_i\|)/log\|t_i\|\|$
`log10_abs_residual()`	`"log_abs"`	$r_i = \|\log\|s_i\| - \log\|t_i\|\|$
`log_abs_residual()`	`"ln_abs"`	$r_i = \|\log_{10}\|s_i\| - \log_{10}\|t_i\|\|$

The following examples illustrate how to use the residual calculators:

```
opt = Fitter("minimize")
opt.set_optimization_parameters(options={'rhobeg':2.0})

# Import factory for residual calculators
from optimizer.targets.cost_functions import residual_generator

# Define target function to accept OptParams
def target(params, residual="signed"):
  res_calc = residual_generator(residual)
  reference = np.array([0.0, 1.0, 2.0, 3.0])
  myfunc = np.array([params[0].value, params[0].value + 1,
                     params[1].value, params[1].value + 1])
  return res_calc(myfunc, reference)

# Define OptParams to be optimized
a = OptParam("a", 1., -2., 2., 1.)
b = OptParam("b", 1., -2., 2., 1.)

# Perform optimization
result = opt.do_fit([a, b], target)

# Print optimization result
print(result)
```

Costs

Cost, or loss, is a single-value aggregate of several residuals used to guide the optimization process. For example, least squares methods typically use the sum of the squares of the residuals as the cost function:

$$c = \sum_i r_i^2$$

The optimizer library provides the following costs that can be set by name using the method *set_cost_calculator*:

cost calculator	name	function
RMSDCost (default)	`"rmsd"`	$c = \sqrt{\frac{1}{W}\sum_i w_i (r_i)^2}$
LeastSquaresCost	`"squared"`	$c = \sum_i w_i (r_i)^2$
L1Cost	`"l1"`	$c = \sum_i w_i \lvert r_i \rvert$
SoftL1Cost	`"soft_l1"`	$c = \sum_i 2w_i * (\sqrt{1 + (r_i)^2}) - 1)$
HuberCost	`"huber"`	$c_i = \begin{cases} (r_i)^2, & r_i \le 1 \\ 2 * \lvert r_i \rvert - 1, & r_i > 1 \end{cases}$ $c = \sum_i w_i c_i$
CauchyCost	`"cauchy"`	$c = \sum_i w_i \log(1 + (r_i)^2)$
ArcTanCost	`"arctan"`	$c = \sum_i w_i \arctan((r_i)^2)$
SumOfResiduals	`"sum"`	$c = \sum_i w_i r_i$
MeanOfResiduals	`"mean"`	$c = \frac{1}{W}\sum_i w_i r_i$

In the table above, w_i are the weights for each residual and $W = \sum_i w_i$.

`"rmsd"` provides the root mean square deviation of the simulation outputs with respect to their target values. The method `"squared"` is equivalent, that is, a minimizer of `"rmsd"` is a minimizer of `"squared"`, with the advantage of smooth derivative at the minimum. On the other hand, its value can grow very fast due to the squared dependence on the individual errors.

`"l1"`, `"soft_l1"` and `"huber"` are similar for large values of the individual errors, which contribute to the cost with a linear function of their absolute value.

Methods `"arctan"` and `"cauchy"` can limit large individual error contributions severely, with a cap to the contribution for the former and with a logarithmic value for the latter.

Note

The methods `"sum"` and `"mean"` simply add the individual residuals and are therefore not positive definite. This means they may not always be appropriate as a measure of error, but they can be useful in situations where a simulation output has to be minimized without reference to a target value, that is, you are only interested in the simulation target achieving the lowest (possibly negative) value.

The following examples illustrate how to change the cost calculator for the Optimizer using the `minimize` backend:

```
opt = Fitter("minimize")
opt.set_optimisation_parameters(jacobian_options={'eps': 1e-3})

# Define target function taking a list of OptParam
def target(params):
  # Jacobian step calculations will pass a 2D array, so they must be handled
  #  when using a gradient-based optimization
  params = np.atleast_2d(params)
  return [[param[0].value**2 + param[1].value**2] for param in params]

# Define OptParams to be optimized
a = OptParam("a", 1., -2., 2., 1.)
b = OptParam("a", 1., -2., 2., 1.)

# Set cost to sum of squares
opt.set_cost_calculator("squared")

# Perform optimization
result = opt.do_fit([a, b], target)

# Print optimization result
print(result)
```

It is also possible to define a custom cost, and in this case it has to be assigned to the `_cost` attribute:

```
# L1Cost calculator
class CustomCost(CostCalculator):
    def evaluate(self, r):
        r = np.asarray(r)
        return self.sum(np.abs(r))

    def derivative(self, r):
        r = np.asarray(r)
        return self.weight_vector((-1.) ** (r < 0))

opt._cost = CustomCost()
# Perform optimization
result = opt.do_fit([a, b], target)
```

Sampling

To explore the parameter space or to create response surface models, it is necessary to create some type of design of experiments (DoE). The most appropriate design depends on the application, and the following sampling methods are provided:

sampling class	description
GridSampling	Creates a grid based (full factorial) design
LatinHypercubeSampling	Creates a latin hypercube design
RandomSampling	Creates a random design
SobolSampling	Creates a design based on Sobol sequences
HaltonSampling	Creates a design based on Halton sequences
SWBDoESampling	Creates a design with one of SWB DoE generator methods.

This classes generate designs in the unit hypercube, which should then be scaled to the problem dimensions.

SWB Design of Experiments generator

The `SWBDoESampling` class provides an interface to the SWB DoE generator, which includes the following classical designs:

- Box–Behnken
- Central composite
- D-optimal
- Fractional factorial
- Hybrid
- Plackett–Burman
- Taguchi

The design can be chose in the constructor using the keyword argument `method`.

Box–Behnken designs

It is selected using `method="box_behnken"`. It does not need any option to be given, the design will only depend on the dimension (number of parameters) selected.

Central composite designs

There are two types of central composite designs that can be used using `method="central_composite"` or `method="small_central_composite".` Small central composite designs are face-centered central composite designs based on a Placket-Burman design instead of on a full factorial, and their implementation is limited for use with up to 8 parameters. No options are required if `method="small_central_composite"` is selected. On the other hand, central composite designs have two parameters that define the location of the points:

- `alpha`: defines the ratio of the radius of the star points to the size of the factorial points in the design. It can be set to `"face centered"`, `"orthogonal"`, `"rotatable"`, or to a floating point value.
- `type`: defines the location of the star points of the design. It can be either `"ccc"` for circumscribed designs or `"cci"` for inscribed designs. In circumscribed designs the factorial points are set to the boundaries of the input domain. For inscribed designs the star points are set to the boundaries of the input domain. This option is only used if alpha is set to `"rotatable"` or to a floating point number.

Example:

```
SWBDoESampling(3, method="central_composite").create_sample(type="cci", alpha="rotatable")
```

D-optimal designs

D-Optimal designs (selected using `method="d_optimal"`) aim to maximize the determinant of the information matrix for the design, and is, therefore, model dependent. The models can be specified as one of the design options:

- `Ns`: desired number of experiments in the final design.
- `model`: polynomial model given as a list of coefficients. It is a list of lists, where each list has as many terms as parameters. For example, [[0, 0, 0], [1, 0, 0], [0, 1, 0], [0, 0, 1]] would represent a full linear model in three parameters.
- `design_matrix`: instead of giving a polynomial model, a design matrix for an arbitrary model can also be given. In this case the option `doe` must be used.
- `doe`: base DoE from which the design points will be selected. If a polynomial model is used, it will default to a two level full factorial design.

Example

For a search using a D-optimal design with seven experiments for three parameters and a quadratic model:

```
SWBDoESampling(3, method="d_optimal").create_sample(**{"NS": 7, "model": [[0 0 0], [1 0 0], [0 1 0], [0 0 1],
[1 1 0], [1 0 1], [0 1 1], [2 0 0], [0 2 0], [0 0 2]]})
```

Fractional factorial designs

It is selected using `method="fractional_factorial"`. Options for fractional factorial designs are:

- `fraction_size`: Fraction size. The size of the base two level full factorial design in dim dimensions is reduced from 2^{dim} to $2^{(dim-fraction_size)}$
- `generator`: Dictionary containing the generator strings for dependent factors
- `alphabet`: String defining the characters that can be used in the generators

For up to 11 parameters there are default designs for selected fraction sizes as shown in the following table:

Dimension	Factor sizes (resolution)
3	1 (III)
4	1 (IV)
5	1 (V), 2 (III)
6	1 (VI), 2 (IV), 3 (III)
7	1 (VII), 2 (IV), 3 (IV), 4 (III)
8	1 (VIII), 2 (V), 3 (IV), 4 (IV)
9	2 (VI), 3 (IV), 4 (IV), 5 (III)
10	3 (V), 4 (IV), 5 (IV), 6 (III)
11	4 (V), 5 (IV), 6 (IV), 7 (III)

If a specific fractional design is desired or a combination that does not have any default design, the generator can be used to define the desired fractional factorial design. For example, for a minimum aberration design for 15 factors and 16 observations of resolution III, the generator dictionary would be:

```
generator = {"e": "ab", "f": "ac", "g": "ad", "h": "bc", "i": "bd",  "j": "cd", "k": "abc", "l": "abd",
             "m": "acd", "n": "bcd", "o": "abcd"}
```

The chracters that can be used to define the generator are defined by the alphabet, which defaults to the ASCII letters, `"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"`. If more than 52 factors have to be referenced then the alphabet with as many characters has to be defined.

Hybrid designs

It is selected using `method="hybrid"`. Options for hybrid designs are:

- `design`: name of the hybrid design It can be one of `"D310"`, `"D311"` (for three parameters), `"D416"` (for four parameters) or `"D628"` (for six parameters).

Plackett–Burman design

It is selected using `method="plackett_burmann"`. Options for Plackett-Burman designs are:

- `Ns`: number of experiments of the desired design. If not given, defaults to $((nparams - 1)//4 + 2) * 4$. If given, it must be a multiple of 4 and larger than the number of parameters.
- `folded` [False]: whether the design will be folded.

Taguchi designs

It is selected using `method="taguchi"`. Options for Taguchi designs are:

- `inner`: design for the inner array. One of `"L4"`, `"L8"`, `"L12"`, `"L16"`, `"L32"` (two level designs), `"L9"`, `"L18"`, `"L27"`, `"L36"`, `"L108"` (three level designs). The number in the design is the number of experiments that the inner array will have.
- `outer`: design for the outer array. If it is used, it can be either `"L4"` or `"L8"`.
- `inner_dims`: Specifies which of the dimensions belong to the inner array when using an outer array.
- `inner_design_cols`: Specifies which columns of the design are used for the inner array when the design accepts more dimensions than the inner array dimension.
- `outer_design_cols`: Specifies which columns of the design are used for the outer array when the design accepts more dimensions than the outer array dimension.

Example

As an example we will generate a Taguchi design for parameters a, b, c, d, e. Three of them (a, b, d) are primary, corresponding to the inner array, and two (c, e) secondary, corresponding to the outer array. The design would be generated as follows:

```
SWBDoESampling(5, method="taguchi").create_sample(**{"inner": "L4", "outer": "L4", "inner_dims": [0, 1, 3]})
```

Other convenience designs

In addition to the previous classical DoEs, some convenience methods are also available:

- `"diagonal"`: generate points in the main diagonal of the input space. The number of points is selected with option `Ns`.
- `"mid_point"`: generate a single point in the middle of the input space.
- `"boundary"`: generate a point in the center of each face of the hypercube defined by the input parameters.
- `"corners"`: generate a point in each corner of the input space. This is equivalent to a two level full factorial design.

Examples

The following examples illustrate how to create a DoE in the unit hypercube and scale it using a `ParamerSpace`:

```
# Grid sampling in 2D, 7x5 grid including boundaries
s = GridSampling(2)
sample = s.create_sample(Ns=(7, 5), boundary=True)

# Design parameters and parameter space
p1 = OptParam("a", 0.5, -1, 1)
p2 = TOptParam("b", 1, 0.1, 100, transform=Log10)

# Create parameter space
ps = ParameterSpace([p1, p2])

# Scale sample to the parameter space with the distribution
#  defined in the scaled space (i.e. uniform in the logarithmic
#  space for p2)
doe_scaled = ps.points_from_unit_doe(sample, scaled=True)
```

Note

The returned sample is a *numpy.ndarray* with shape `(ndim, npoints)`.

Sensitivity and Screening Analysis

Sensitivity analysis can be used to understand how the variability of parameters affects the behavior of the responses. There are two types of sensitivity analysis that can be performed: one-factor-at-a-time (OAT) and the method of elementary effects (EE). OAT sensitivy analysis is mainly used for a local study around a nominal point, whereas EE sensitivity analysis is more useful to consider global properties of the system.

A common use for Sensisitivity Analysis is the determination of the subset of parameters with the largest impact on the problem. In this case we can talk of Screening Analysis. In this case the information is used to determine how much the different parameters affect the simulation responses and rank them according to their influence on each response, which can then be used to select a subset of parameters.

Local Sensitivity Analysis

Local Sensitivity Analysis and, in particular, one-factor-at-a-time sensitivity analysis aims at understanding the response of the model as a function of small changes to a single parameter, with all other parameters fixed. Therefore, it reveals only the local changes of the response surface of the model with respect to a given parameter. Since the number of simulations only increases linearly with the number of parameters, it can be provide a quick way to analyse the influence of different paramters and, if more than two points are simulated in each direction, also their nonlinearity around the nominal point.

The sensitivity of response f^k to parameter i is calculated as

$$s_{ik} = \frac{100}{p_r} \frac{\sum_j |f^k(\boldsymbol{x}^c) - f^k(\boldsymbol{x}^{i,j})| / |x_i^{i,c} - x_i^{i,j}|}{\sum_j 1/|x_i^{i,c} - x_i^{i,j}|} = \frac{100}{p_r} \sum_j \omega_j^i |f^k(\boldsymbol{x}^c) - f^k(\boldsymbol{x}^{i,j})|,$$

where p_r is the selected percentage range used to the analysis, c is the midpoint level, j runs over all levels $j \neq c$, $\boldsymbol{x}^{i,j}$ is the point for level j of parameter i, and $x_i^{i,j}$ is the coordinate i of the point for level j of parameter i. ω_j^i is , for a given direction i, a weight associated to each of the points of the design, and it is given by $\omega_j^i = 1/\sum_k |x_i^{i,c} - x_i^{i,j}| / |x_i^{i,c} - x_i^{i,k}|$, so it is inversly proportional to the distance of the point $x_i^{i,j}$ to the midpoint x_i^c.

Note

Including the normalization factor $\frac{100}{p_r}$ allows for an easier comparison of the sensitivities for the same function with different values of the parameters. For example, for a target close to linearity, the normalization results in a similar value of the sensitivity regardless of `percentage_range`.

Note

If the direction is chosen as `backward` or `forward`, then the sensitivity can take signed values, with positive ones representing an increase in the target and negative ones, a decrease in the target.

For a single level and a single response,

$$s_i = \frac{100}{p_r} \frac{|f(\boldsymbol{x}^c) - f(\boldsymbol{x}^{i,low})| / |x_i^{i,c} - x_i^{i,low}| + |f(\boldsymbol{x}^c) - f(\boldsymbol{x}^{i,high})| / |x_i^c - x_i^{i,high}|}{1/|x_i^{i,c} - x_i^{i,low}| + 1/|x_i^{i,c} - x_i^{i,high}|}$$

If the midpoint level is equidistant to both the low and high levels and $p_r = 100$,

$$s_i = \frac{1}{2}\left[|f(\boldsymbol{x}^c) - f(\boldsymbol{x}^{i,low})| + |f(\boldsymbol{x}^c) - f(\boldsymbol{x}^{i,high})|\right]$$

The relative sensitivity is defined as a percentage:

$$s_{r,ik} = 100 \times \frac{s_{ik}}{|f^k(\boldsymbol{x}^c)|}$$

Nonlinearity

When using Local Sensitivity Analysis when more than three points, a measure of nonlinearity in the selected range will also be given. Since the local sensitivity composed from linear measures, the higher the nonlinearity, the more dependent the sensitivity is to the selected range.

The nonlinearity λ of the function f is defined as the RMSD between f and a linear approximation to it, $\hat{f}$, normalised by the range of the function in the domain of the design,

$$\lambda = \frac{1}{\Delta f|_{x_L}^{x_R}} \sqrt{\frac{\int_{x_L}^{x_R} (f(x) - \hat{f}(x))^2 \, dx}{x_R - x_L}} = \frac{RMSD(f;\hat{f})}{\Delta f|_{x_L}^{x_R}},$$

where x_L, x_R are the domain boundaries, $\Delta f|_{x_L}^{x_R} = \mathbf{max}(f)|_{x_L}^{x_R} - \mathbf{min}(f)|_{x_L}^{x_R}$ and $\mathbf{max}\,/\,\mathbf{min}(f)|_{x_L}^{x_R}$ is the maximum/minimum of f in $[x_R, x_L]$.

Note

The functions $f(x)$ and $\hat{f}(x)$ are approximated using the sampling performed for the sensitivity analysis, so increasing the value of parameter npoints also increases the accuracy of the calculated nonlinearity.

Example

Consider the following problem:

$$err = |a + b| \; + \; a^2 + b^2 + |ab| - c^2 d$$
$$err2 = c^2 d$$

Prameters `a` and `b` affect variable `err` but not `err2`. Parameters `c` and `d` affect both targets.

First, we define a function with interface as expected from sensitivity_analysis:

```
def func(params, return_individual_errors=False, verbose=False):
  """
  - The callable will receive a list of list of OptParam. The outer lists has as many elements
    as number of points to be evaluated. The inner lists as many elements as number of parameters
  - The keyword argument 'return_individual_errors' will be passed with value True. It is used to
    tell the function to return errors individually (instead of an aggregate)
  """
  # 1. Make an array to calculate the function
  npoints = len(params)
  ndim = len(params[0])
  x = np.empty((npoints, ndim))
  for i, params_i in enumerate(params):
    x[i, :] = [p.value for p in params_i]

  # 2. Calculate errors. Functions are expected to return (npoints, nouts) array. E.g nouts=10 with 5 outputs
from low Vg and 5 outputs from high Vg
  #    In this example nouts = 2: first column=\left|a+b\right|\ +\ a^2+b^2+\left|ab\right|-c^2d, second
column=c^2d
  err = np.abs(x[:, 0] + x[:, 1]) + x[:, 0]**2 + x[:, 1]**2 + np.abs(x[:, 0] * x[:, 1]) - x[:, 2]**2 * x[:, 3]
  err2 = x[:, 2]**2 * x[:, 3]
  errors = np.hstack((err.reshape(-1, 1), err2.reshape(-1, 1)))

  if return_individual_errors:
    # 3. To have the screening of individual contributions the function is expected to return
    #    a) The array with all the errors for each point , shape=(npoints, nouts)
    #    b) one list with as many elements as points, where each element is
    #       a list of tuples of the form ("error_name", error_vector), with error_vector
    #       is a subset of the total error vector assigned to source "error_name". For example
    #       a list with npoints elements with each element [("error_logvg", error_vetor_lowvg),
("error_highvg", error_vetor_highvg)],
    #       where len(error_vetor_lowvg)=len(error_vetor_highvg)=5. In general, the sum of the lengths of
individual error_vector should be nouts
    #       In this example, it could be one tuple with len(error_vector)=2 or two tuples with
len(error_vector)=1
    ierrors = [[("err", [err[i]]), ("err2", [err2[i]])] for i in range(x.shape[0])]
    """
    Example output:

    errors is a npoints x nout numpy array. In this case npoints=5 (2 params * 2 off centre values per param +
1 centre value), nouts=2
    errors = [[-2.42388074,  5.06274073],
              [ 4.80371259,  2.26570711],
              [ 2.42351962,  0.27576303],
              [ 3.92500296,  0.71495111],
              [-0.3220704 ,  1.16088707]]

    ierrors also has npoints elements. Each element is a list with a tuple per individual component of the
total error. The sum of the lengths of all individual components should be = nout
    ierrors = [[('err', [-2.4238807398279896]), ('err2', [5.062740732936316])],
              [('err', [4.803712591858723]), ('err2', [2.2657071078770907])],
              [('err', [2.4235196224815674]), ('err2', [0.27576302763718086])],
              [('err', [3.925002961366277]), ('err2', [0.7149511082951449])],
              [('err', [-0.322070395350492]), ('err2', [1.1608870685827413])]]

    """
    return errors, ierrors
  return errors
```

Note

The target function for screening analysis must be able to accept the keywords *return_individual_errors* and *verbose*, since these will be passed internally every time it is called.

Note

The individual errors are required to separate different components of the output. Even though the output only has one component this is still required.

Next, we define the optimisation parameters for this problem with *a*, *b*, *c*, *d* in *[0, 2]*:

```
a = OptParam('a', 1, 0, 2)
b = OptParam('b', 1, 0, 2)
c = OptParam('c', 1, 0, 2)
d = OptParam('d', 1, 0, 2)
```

Finally, the sensitivity of function *func* to parameters *a*, *b*, *c* and *d* is calculated as:

```
npoints=7
s = sensitivity_analysis([a, b, c, d], func, percentage_range=10, npoints=7, disp=True)
```

This produces the following sensitivity analysis report:

```
Sensitivity Analysis using 10% of the total parameter ranges:
    a: (0.9, 1.1)
    b: (0.9, 1.1)
    c: (0.9, 1.1)
    d: (0.9, 1.1)


Deviation from linearity for each target:
+-----------+----------------------+-----------------------+-------------------------+
| Parameter | Nonlinearity [%]: err | Nonlinearity [%]: err2 | Nonlinearity [%]: total |
+-----------+----------------------+-----------------------+-------------------------+
|     a     |         0.16         |          nan          |          0.18           |
|     b     |         0.16         |          nan          |          0.18           |
|     c     |         0.32         |          0.32         |          0.07           |
|     d     |         0.00         |          0.00         |          0.16           |
+-----------+----------------------+-----------------------+-------------------------+

Sensitivity Analysis per target:
+-----------+--------------------+----------------------+-------------------+-----------------------+
| Parameter |  Sensitivity: err  | Sensitivity [%]: err | Sensitivity: err2 | Sensitivity [%]: err2 |
+-----------+--------------------+----------------------+-------------------+-----------------------+
|     a     | 2.1818181818181825 |  54.54545454545456   |        0.0        |          0.0          |
|     b     | 2.1818181818181817 |  54.54545454545454   |        0.0        |          0.0          |
|     c     | 1.0909090909090895 |  27.272727272727238  | 1.090909090909092 |   109.09090909090922  |
|     d     | 0.5454545454545442 |  13.636363636363605  | 0.545454545454546 |   54.54545454545461   |
+-----------+--------------------+----------------------+-------------------+-----------------------+

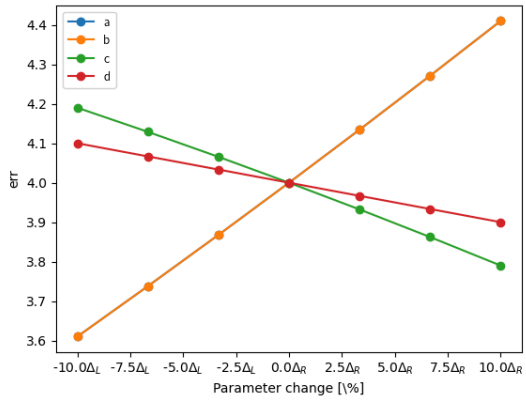
Sensitivity Analysis on total rmsd of all targets:
+-----------+--------------------+--------------------+
| Parameter |    Sensitivity     |  Sensitivity [%]   |
+-----------+--------------------+--------------------+
|     a     | 1.4966140472085832 | 51.33343832013518  |
|     b     | 1.4966140472085827 | 51.33343832013517  |
|     c     | 0.559334748609666  | 19.185023601519717 |
|     d     | 0.2805711263746842 | 9.623510241019675  |
+-----------+--------------------+--------------------+
```

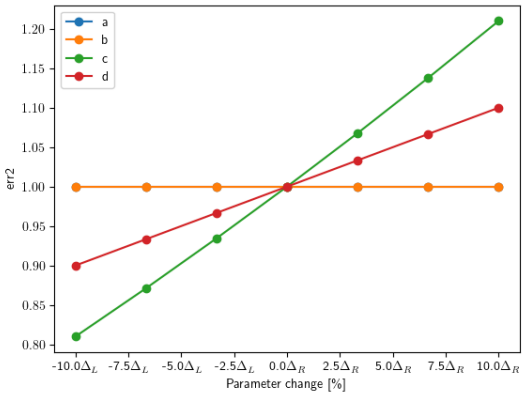
The first table reports nonlinearity if more than 3 points per parameter are simulated. In this case individual nonlinearities for `d` are zero because of the linear dependence, but when combining the errors the dependence loses its linearity. The *nan* entries correspond to the lack of any dependence on the parameter.

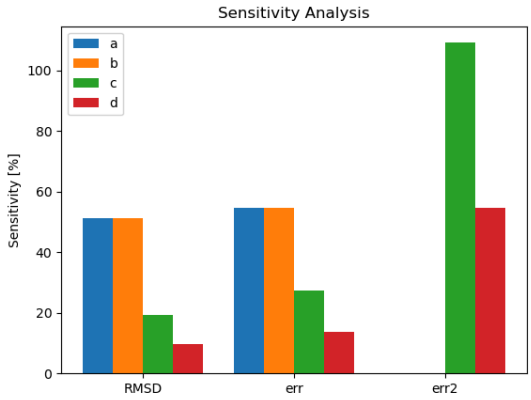
The next two tables report the sensitivity. The first one has two columns per target, the first column contains the sensitivity in the same units as the target, and the second column contains the value as a percentage with respect to the midpoint value.

The last table summarizes the sensitivities for all targets using the root-mean-square deviation (RMSD) of all individual sensitivities.

The following plots are also created if a plotting directory (*plot_dir*) is given:







The first two plots show the evaluation of each target when changing each parameter. In this case, due to the small range of parameter used, the dependence looks linear, but the values in the nonlinearity table show that they are in fact not linear.

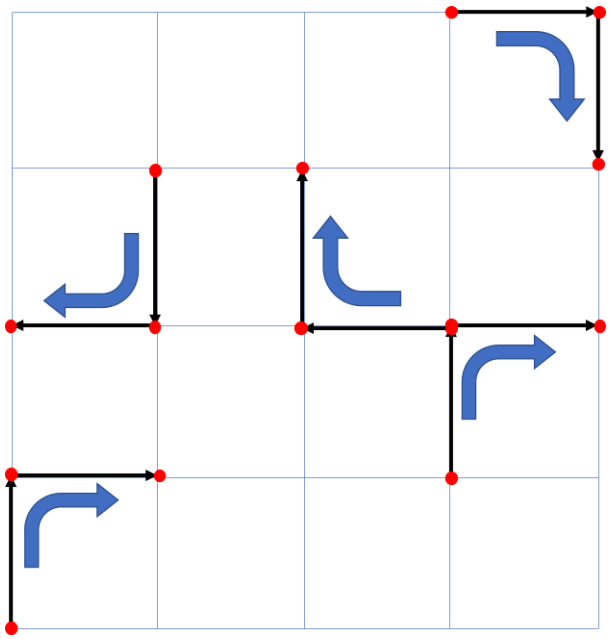
The last plot shows the sensitivities for each target and parameter for easy comparison of the impact of each parameter on each target.

Global Sensitivity Analysis

In contrast to local sensitivity analysis, global sensitivity analysis aims to summarize information of the complete domain, including interactions between parameters to some extent. The implemented methodology, the method of Elementary Effects [Morris1991], [Campolongo2007] is based on aggregating data from multiple one-factor-at-a-time experimental designs or trajectories. There are two parameters used to define the analysis. `nlevels` determines how many levels each of the parameters can take. For example, if there are four parameters in the analysis and `nlevels=5`, then the total number of possible parameter value combinations is $5^4 = 625$ (these are the possible values, not the actually simulated values). The other parameter needed to define the analysis is `ntrajectories`, which determines how many trajectories are executed. Each of this trajectories is local in the sense that:

1. Each trajectory has as many points as parameters in the study plus one. For example, for four parameters, each trajectory has 5 points.
2. In each trajectory, the coordinates can differ from the previous point in the trajectory in only one dimension and only by one step in the grid defined by `nlevels`.
3. In each trajectory, all dimensions must be changed exactly once.

The following figure shows an example of five trajectories in two dimensions using five levels for each parameter.



For each trajectory j we can calculate the value of the elementary effect for target function k in each dimension i:

$$EE_{ik}^{j} = \frac{f_k(x^{i+1,j}) - f_k(x^{i,j})}{\Delta}$$

where $x^{i,j}$ and $x^{i+1,j}$ denote the initial and final point of the trajectory j's edge changing the value of parameter i. $\Delta = x_i^{i+1,j} - x_i^{i,j}$ is the step in the parameter and $f_k(x^{i,j})$ denotes the value of target function k at $x^{i,j}$.

The mean (over trajectories), mean of the absolute and standard deviation of the values EE_{ik}^{j} can be used as sensitivty metrics to quantify the inflence of each parameter:

$$\mu_{ik} = \frac{1}{N_t} \sum_j EE_{ik}^j$$

$$\mu_{ik}^* = \frac{1}{N_t} \sum_j |EE_{ik}^j|$$

$$\sigma_{ik} = \sqrt{\frac{1}{N_t - 1} \sum_j \left(EE_{ik}^j - \mu_{ik}\right)^2}$$

In particular, μ^* can be used as the main metric to determine the total (i.e. including interactions) influence of each parameter. The standard deviation of the elementary effects σ_{ik} gives additional information regarding the variability of the dependence across the space. For example, if σ_{ik} is very small it means that the dependence of the target k on the parameter i is uniform across the space but if σ_{ik} is large, then it is an indication that the influence of the target k on the parameter i is highly dependent on the location in the parameter space.

Morris1991

Morris, M. D. (1991). Factorial Sampling Plans for Preliminary Computational Experiments. Technometrics, 33(2), 161–174.

Campolongo2007

Campolongo, F., Cariboni, J. and Saltelli, A. (2007). An effective screening design for sensitivity analysis of large models. Environmental Modelling and Software, 22(10), 1509–1518.

Example

For the same problem used for local sensitivity analysis,we run a global sensitivity analysis using the following command:

```
s = elementary_effects([a, b, c, d], func, nlevels=5, ntrajectories=10)
```

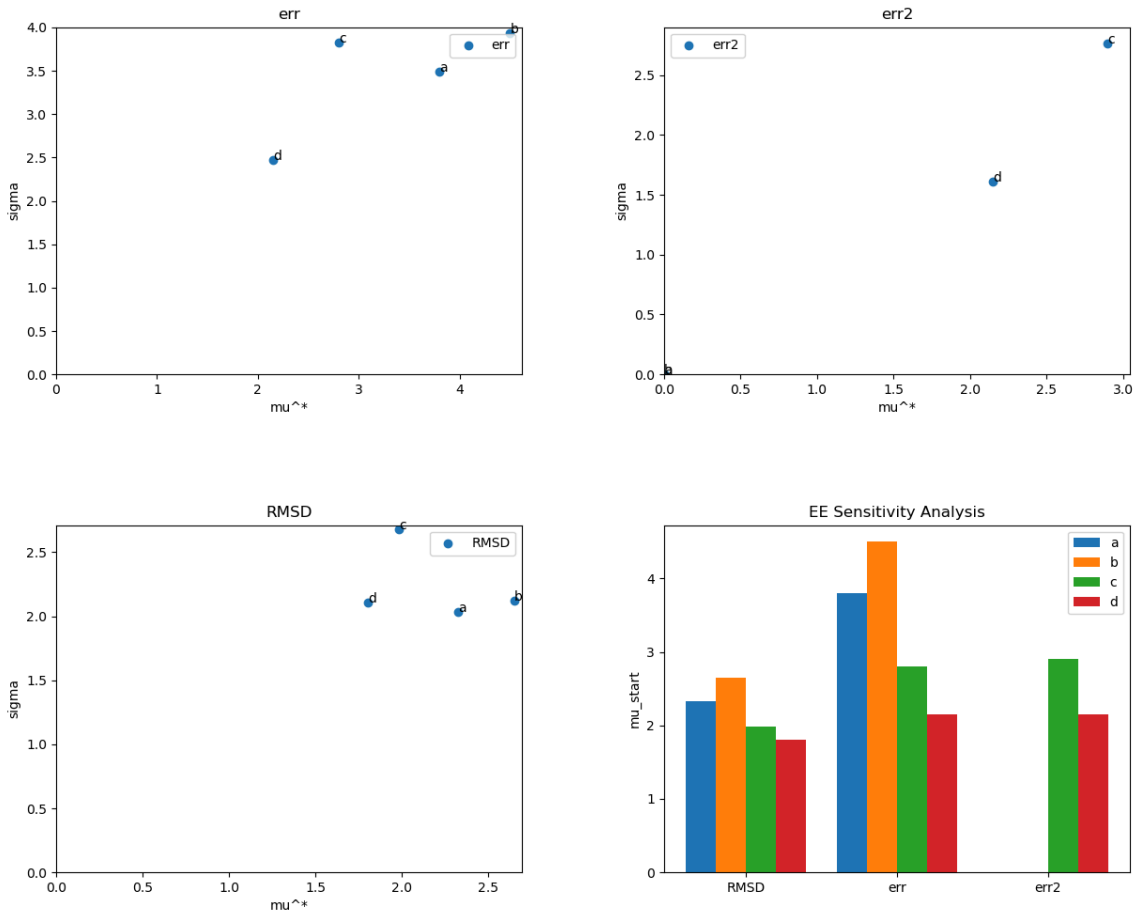
The corresponding sensitivity analysis report is:

```
Elementary effects per target:
+-----------+---------+--------------------+-----------+----------+--------------------+------------+
| Parameter | mu: err |     sigma: err     | mu^*: err | mu: err2 |    sigma: err2     | mu^*: err2 |
+-----------+---------+--------------------+-----------+----------+--------------------+------------+
|     a     |   3.6   |  2.0591260281974   |    3.9    |   0.0    |        0.0         |    0.0     |
|     b     |   2.65  | 2.933001875212493  |    3.55   |   0.0    |        0.0         |    0.0     |
|     c     |   0.85  | 3.1488092987667575 |    2.3    |   2.45   | 2.4464259645450133 |    2.45    |
|     d     |   0.1   | 1.6926310879810755 |    1.15   |   1.3    | 1.2786711852544423 |    1.3     |
+-----------+---------+--------------------+-----------+----------+--------------------+------------+

Elementary effects on total rmsd of all targets:
+-----------+---------------------+--------------------+--------------------+
| Parameter |          mu         |       sigma        |        mu^*        |
+-----------+---------------------+--------------------+--------------------+
|     a     |  2.282598272180992  | 1.4181192836975636 |  2.42325960993621  |
|     b     |  1.7760387066532217 | 1.829950698564889  | 2.060933860370963  |
|     c     |  1.3251566051191617 | 2.6662587418977504 | 1.8171682082119625 |
|     d     | 0.46183248408397015 | 1.3343441143722794 | 0.9087959987980365 |
+-----------+---------------------+--------------------+--------------------+
```

In the case of global sensitivity analysis we get two tables. The first one summarizes the sensitivity measures for each target and the seond one for the RMSD of all targets. In each case there are three columns for each target containing the previously defined measures μ, μ^* and σ.

The graphics below are generated as the visual summary of the sensitivity analysis.



The top two figures show the values for μ^* and σ for all parameters and for each target. The location along the μ^*-axis gives a measure of the influence of the parameter, while the location along the σ-axis gives a measure of the variability of the elementary effects across the complete space. The bottom-left panel shows the same for the RMSD of all targets and the bottom-right panel gives a visual overview of μ^* for all parameters and targets.

Screening Analysis with Polynomial RSM

For some problems with many parameters you might be interested in selecting only a subset with the largest impact. You can use screening analysis to determine how much the different parameters affect the simulation responses. Parameters are then ranked according to their influence on each response, and this information can be used to select a subset of parameters. Due to the possibly large number of parameters, this task uses a polynomial response surface model (RSM) built from a given DOE such as a Plackett–Burman design. These RSMs are used to obtain the effect of each parameter on the simulation responses. This is done using the normalized coefficients β of the RSMs build from the normalized parameters $\hat{x}_i$ to

rank parameters according to their effect on the responses. Using a polynomial model across the design space allows taking into account interactions between parameters, but it requires the data to be well approximated by the model.

$$\hat{y}_j = \beta_0 + \beta_{x_0}\hat{x}_0 + \ldots + \beta_{x_N}\hat{x}_N + \beta_{x_0x_1}\hat{x}_0\hat{x}_1 + \ldots$$

$$\text{influence}_{x_i \ldots x_j} = \frac{\beta_{x_i \ldots x_j}}{\sum_{k \neq 0} \beta_k}$$

Note

If two parameters are influential through their interaction they will both be selected.

Example

Once again, we will consider the same problem used for local sensitivity analysis:

$$\begin{aligned} err &= |a+b| \; + \; a^2 + b^2 + |ab| - c^2 d \\ err2 &= c^2 d \\ a, b, c, d &\in [0, 2] \end{aligned}$$

Once again, we notice that parameters `a` and `b` have an impact on the variable `err` but not `err2`. In addition, parameters `c` and `d` are not coupled to `a` and `b` in any of the targets. A second-order response surface model built from a grid design will be used for the analysis.

We can call the screening analysis for function *func* (as defined above for local sensitivity analysis) and parameters *a*, *b*, *c*, *d*:

```
sa_result = screening_analysis([a, b, c, d], func,
                               doe="grid",
                               doe_params={"Ns": 3},
                               order=2,
                               screening_criterion="local_strong",
                               screen_best=3)
```

The returned *ScreeningAnalysisResult* object `sa_result` contains a list of parameters in `selected_parameters` that can be used in any subsequent call to optimization routines. The call will also produce the following report:

```
Statistics for linear model regressions:
+--------------------+--------------------+----------------------+--------------------+
|         r2         |       r2Adj        |         MSE          |        AICc        |
+--------------------+--------------------+----------------------+--------------------+
| 0.9988929250994638 | 0.9986580910296531 | 0.015962090036164125 | 42.72139609226021 |
| 0.9937803692905733 | 0.9924610536855435 | 0.01596209003616412  | 42.72139609226018 |
+--------------------+--------------------+----------------------+--------------------+

Screening Analysis - screening 3 most influential terms
+-------+------------------------+----------+
|  Term |     Influence [%]      | Selected |
+-------+------------------------+----------+
| c x d |   21.663937523617562   |   True   |
|   c   |   21.66393752361756    |   True   |
|   d   |   14.041440987529908   |   True   |
|   b   |   11.563169164882222   |  False   |
|   a   |   11.56316916488222    |  False   |
| c x c |   10.831968761808762   |  False   |
| a x a |   2.8907922912205923   |  False   |
| a x b |   2.8907922912205573   |  False   |
| b x b |   2.890792291220556    |  False   |
| d x d |  4.08493619331039e-14  |  False   |
| a x d | 1.517969870518788e-14  |  False   |
| b x d | 6.627999238221833e-15  |  False   |
| a x c | 6.1578466220821166e-15 |  False   |
| b x c |  2.42453116407111e-15  |  False   |
+-------+------------------------+----------+
```

The first table reports the results of the polynomial fitting for each target. The report includes the coefficient of determination (`r2`) and the adjusted coefficient of determination (`r2Adj`) to quantify the quality of the fit, the mean square error (`MSE`) of the fit, and the corrected Akaike information criterion (`AICc`) to compare different models (for example, RSM of orders 1 and 2).

The second table includes a summary of the importance of each term in the RSM used. In this example, `screen_best=3` was used, so only the top three parameters have been selected. This would translate to parameters `c` and `d` since the third most influential term is an interaction between them. In this case, there are no terms containing `d x d` , `a x c` , `a x d` , `b x c` , or `b x d` terms in the models, so the RSM correctly reports that they do not have any influence.

Response Surface Models

Response surface model (RSM) objects provide a convenient interface to create RSM from available data (e.g. from simulation done at some given DoE points). There are different types of supported RSMs such as polynomials and Gaussian processes that can be appropriate depending on the nature of the data.

Examples

The following example illustrates the creation of a polynomial response surface object to represent the data stored in the arrays *x* and *y*:

```
# Parameters that describe the space of the problem
p1 = OptParam("p1", 0.5, 0, 1)
p2 = OptParam("p2", 0.5, 0, 1)

# Data from some simulation
x = np.random.uniform(0, 1, size=(10, 2))
y = np.sin(x[:, 0])*np.sin(x[:, 1])

# Define a quadratic polynomial RSM for the space defined by p1 and p2
poly = PolynomialRSM([p1, p2], order=2)

# Create the model from the data
poly.create(x, y)

# Print summary of the fit quality
print(poly.fit_summary)

# Test at some additional points
x = np.random.uniform(0, 1, size=(10, 2))
y = np.sin(x[:, 0])*np.sin(x[:, 1])

r2 = poly.score(x, y)
print(f"r2 = {r2}")
```

If instead of a polynomial a Gaussian process is more appropriate:

```
# gaussian process RSM for the space defined by p1 and p2
gp = GaussianProcessRSM([p1, p2])

# Create the model from the data
gp.create(x, y)

# Test at some additional points
x = np.random.uniform(0, 1, size=(10, 2))
y = np.sin(x[:, 0])*np.sin(x[:, 1])

r2 = gp.score(x, y)
print(f"r2 = {r2}")
```

Gaussian Process Kernels

Gaussian process RSMs used to fit an n-dimensional Gaussian process to the provided targets are characterized by a kernel that determines the smoothness of the approximating surface. By default, the kernel used is *Matern32*, which is one time differentiable and therefore does not enforce too much smoothness. For a two times differentiable version, *Matern52* could be used and for an infinitely differentiable kernel, *SquaredExponential* would be appropriate.

The available kernels are:

kernel	parameters	description
Matern	`l_scale`	characteristic length scale
	`nu`	order of the kernel
Matern32	`l_scale`	characteristic length scale
Matern52	`l_scale`	characteristic length scale
NeuralNetwork	`sigma`	variance parameter for input
	`sigma_0`	variance parameter for constant term
RationalQuadratic	`alpha`	rational exponent
	`l_scale`	characteristic length scale
SquaredExponential	`l_scale`	characteristic length scale

Examples

The following examples illustrate how to create a GP RSM with different kernels in one dimension:

```
# 1D example
x = np.random.uniform(size=(10, 1))
y = ((x-0.5)**2).ravel()

p1 = OptParam("p1", 0.5, 0, 1)

# Fit Gaussian process with Matern52 kernel
kernel = Matern52(l_scale=1)
model = GaussianProcessRSM([p1], kernel=kernel)
model.create(x, y)

x_pred = np.linspace(1e-2, 1, 100).reshape((-1, 1))
pred = model(x_pred)

# Fit Gaussian process with SquaredExponential kernel
kernel = SquaredExponential(l_scale=1)
model = GaussianProcessRSM([p1], kernel=kernel)
model.create(x, y)

pred = model(x_pred)

# Fit Gaussian process with Matern 7/2 kernel
kernel = Matern(nu=3.5, l_scale=1)
model = GaussianProcessRSM([p1], kernel=kernel)
model.create(x, y)

pred = model(x_pred)

# Fit Gaussian process with Matern52 kernel
kernel = NeuralNetwork(sigma_0=1, sigma=1)
model = GaussianProcessRSM([p1], kernel=kernel)
model.create(x, y)

pred = model(x_pred)
```

For higher dimensions it is equivalent, but some kernel parameters can be specified as arrays:

```
#2D example
# Parameters that describe the space of the problem
p1 = OptParam("p1", 0.5, 0, 1)
p2 = OptParam("p2", 0.5, 0, 1)

# Data from some simulation
ss = GridSampling(2)
x = ss.create_sample(Ns=(7, 5)).T
y = np.sin(2*np.pi*x[:, 0])*np.sin(x[:, 1])

# Test at some additional points
xtest = np.random.uniform(0, 1, size=(10, 2))
ytest = np.sin(2*np.pi*xtest[:, 0])*np.sin(xtest[:, 1])

# Fit Gaussian process with SquaredExponential kernel. Length scale in both dimensions is the same
kernel = SquaredExponential(l_scale=1)
model = GaussianProcessRSM([p1, p2], kernel=kernel)
model.create(x, y)

# Evaluate goodness of fit
gof = model.score(xtest, ytest)

# Fit Gaussian process with SquaredExponential kernel. Length scales in both dimensions are independent
kernel = SquaredExponential(l_scale=[1, 1])
model = GaussianProcessRSM([p1, p2], kernel=kernel)
model.create(x, y)

# Evaluate goodness of fit
gof = model.score(xtest, ytest)
```

If a different type of kernel is appropriate for different dimensions, it can be specified using kernel multiplication or addition and the keyword `dimensions`:

```
# Fit Gaussian process with SquaredExponential kernel in dimension 0 and NeuralNetwork kernel in dimension 1
kernel = SquaredExponential(l_scale=1, dimensions=(0,)) * NeuralNetwork(sigma_0=1, sigma=1, dimensions=(1,))
model = GaussianProcessRSM([p1, p2], kernel=kernel)
model.create(x, y)

# Evaluate goodness of fit
gof = model.score(xtest, ytest)
```

Note

Depending on the training data, the fit of kernel parameters may result in unphysical models. In this case, the parameters can be kept fixed to their initial value passing the option `hyperparam_mode="fixed"` in the creation of the *GaussianProcessRSM*

Copyright

Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader’s responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

https://www.synopsys.com